Exercise1_1

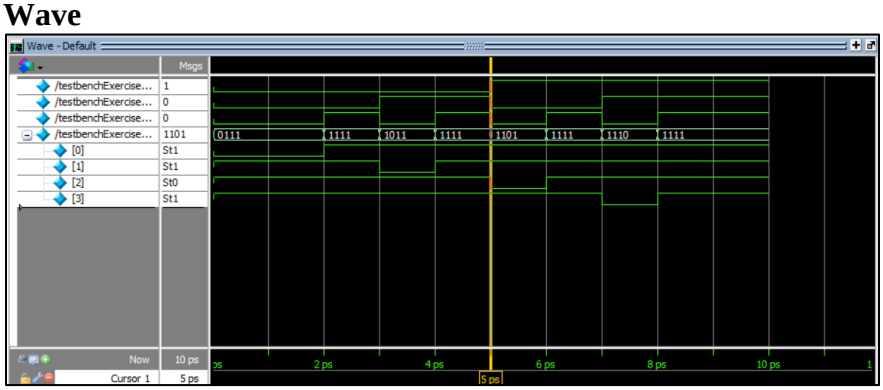
HDL Code

```
module exercise1_1(W,X,Y,Z);

        output [0:3]W;
        input X, Y;
        input Z;
        wire X1, Y1, Z1;

        not G1(X1,X), G2(Y1, Y), G3(Z1,Z);

        nand G4(W[0],X1,Y1,Z1),G5(W[1],X1,Y,Z1),G6(W[2],X,Y1,Z1),G7(W[3],X,Y,Z1);

endmodule

module testbenchExercise1_1;

        reg X,Y,Z;
        wire [0:3]W;
        exercise1_1 tb1(W,X,Y,Z);

        initial begin

                X=1'b0; Y=1'b0; Z=1'b0;
                $display("Simulating output for Exercise1_1");
                $display("ps\tW\tX\tY\tZ");
                $monitor("%0t\t%b\t%b\t%b\t%b",$time,W,X,Y,Z);

                #2 X=1'b0; Y=1'b0; Z=1'b1;
                #1 X=1'b0; Y=1'b1; Z=1'b0;
                #1 X=1'b0; Y=1'b1; Z=1'b1;
                #1 X=1'b1; Y=1'b0; Z=1'b0;
                #1 X=1'b1; Y=1'b0; Z=1'b1;
                #1 X=1'b1; Y=1'b1; Z=1'b0;
                #1 X=1'b1; Y=1'b1; Z=1'b1;
                #2 $finish;

        end

endmodule
```

Transcript

```
VSIM 23> run
# Simulating output for Exercise1_1
# ps    W       X       Y       Z
# 0     0111    0       0       0
# 2     1111    0       0       1
# 3     1011    0       1       0
# 4     1111    0       1       1
# 5     1101    1       0       0
# 6     1111    1       0       1
# 7     1110    1       1       0
# 8     1111    1       1       1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Exercise1_1.v(34)
#    Time: 10 ps  Iteration: 0  Instance: /testbenchExercise1_1
# 1
# Break in Module testbenchExercise1_1 at C:/intelFPGA_lite/20.1/Exercise1_1.v line 34
```

Wave



Exercise1_2

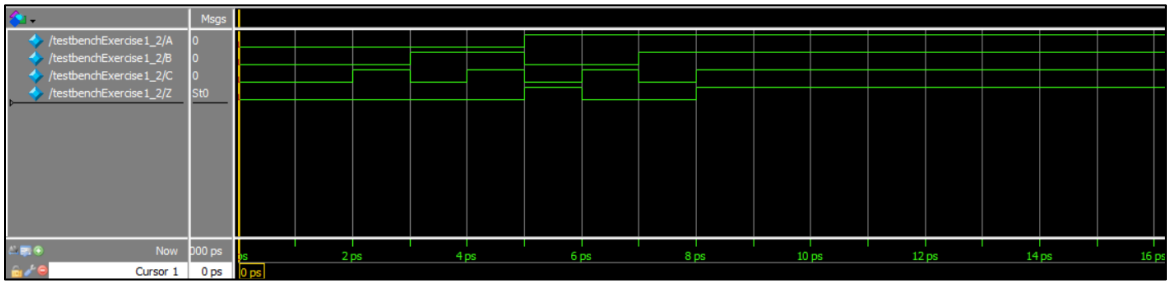
HDL Code

```
module exercise1_2(A,B,C,X);

        input A, B, C;
        output X;
        wire wire1, wire2, wire3;

        not NOT(wire2, A);
        xor EOR2(wire1, B, C);
        and AND2(wire3, wire1, A);
        nor NOR2(X, wire3, wire2);

endmodule

module testbenchExercise1_2;

        reg A, B, C;
        wire Z;
        exercise1_2 tb1(A, B, C, Z);

        initial begin
                $display("Simulating output for Exercise1_2");
                $display("ps\tA\tB\tC\tZ");

                #0      A = 1'b0; B = 1'b0; C = 1'b0;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #2      A = 1'b0; B = 1'b0; C = 1'b1;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #1      A = 1'b0; B = 1'b1; C = 1'b0;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #1      A = 1'b0; B = 1'b1; C = 1'b1;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #1      A = 1'b1; B = 1'b0; C = 1'b0;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #1      A = 1'b1; B = 1'b0; C = 1'b1;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #1      A = 1'b1; B = 1'b1; C = 1'b0;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #1      A = 1'b1; B = 1'b1; C = 1'b1;
                #0      $display("%0t\t%b\t%b\t%b\t%b", $time, A, B, C, Z);
                #79992  $finish;
        end

endmodule
```

Transcript

```
VSIM 20> run
# GetModuleFileName: The specified module could not be found.
#
#
# Simulating output for Exercise1_2
# ps    A       B       C       Z
# 0     0       0       0       0
# 2     0       0       1       0
# 3     0       1       0       0
# 4     0       1       1       0
# 5     1       0       0       1
# 6     1       0       1       0
# 7     1       1       0       0
# 8     1       1       1       1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Exercise1_2.v(40)
#    Time: 80 ns  Iteration: 0  Instance: /testbenchExercise1_2
# 1
# Break in Module testbenchExercise1_2 at C:/intelFPGA_lite/20.1/Exercise1_2.v line 40
```

Wave



Exercise1_3

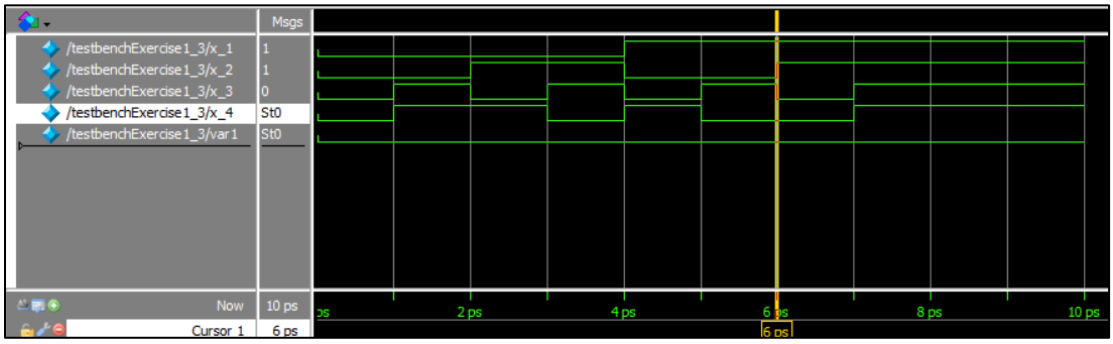
HDL Code

```
module exercise1_3(var1,x_4,x_1,x_2,x_3);

        output var1, x_4;
        input x_1, x_2, x_3;
        xor EOR1(x_4,x_1,x_2,x_3);
        xor EOR2(var1,x_1,x_2,x_3,x_4);

endmodule

module testbenchExercise1_3;

        reg x_1, x_2, x_3;
        wire var1, x_4;

        exercise1_3 tb1(var1,x_4,x_1,x_2,x_3);

        initial begin
                x_1=1'b0; x_2=1'b0; x_3=1'b0;
                $display("Simulating output for Exercise1_3");
                $display("ps\tx_1\tx_2\tx_3\tx_4\tvar1");
                $monitor("%0t\t%b\t%b\t%b\t%b\t%b",$time,x_1,x_2,x_3,x_4,var1);

                #1 x_1=1'b0; x_2=1'b0; x_3=1'b1;
                #1 x_1=1'b0; x_2=1'b1; x_3=1'b0;
                #1 x_1=1'b0; x_2=1'b1; x_3=1'b1;
                #1 x_1=1'b1; x_2=1'b0; x_3=1'b0;
                #1 x_1=1'b1; x_2=1'b0; x_3=1'b1;
                #1 x_1=1'b1; x_2=1'b1; x_3=1'b0;
                #1 x_1=1'b1; x_2=1'b1; x_3=1'b1;
                #3 $finish;
        end

endmodule
```

Transcript

```
VSIM 107> run
# GetModuleFileName: The specified module could not be found.
#
#
# Simulating output for exercise1_3
# ps    x_1     x_2     x_3     x_4     var1
# 0     0       0       0       0       0
# 1     0       0       1       1       0
# 2     0       1       0       1       0
# 3     0       1       1       0       0
# 4     1       0       0       1       0
# 5     1       0       1       0       0
# 6     1       1       0       0       0
# 7     1       1       1       1       0
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Exercise1_3.v(40)
#    Time: 10 ps  Iteration: 0  Instance: /testbenchExercise1_3
# 1
# Break in Module testbenchExercise1_3 at C:/intelFPGA_lite/20.1/Exercise1_3.v line 40
```

Wave



Drill1_1

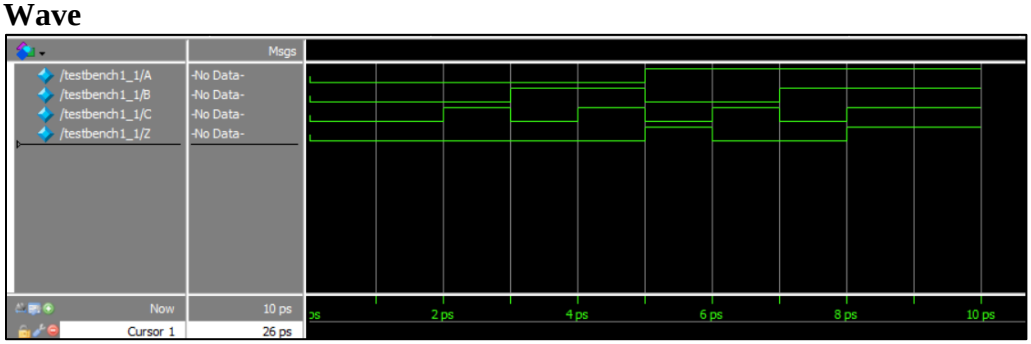
HDL Code

```
module circuit1_1(A,B,C,X);

        input A, B, C;
        output X;
        wire wire1, wire2, wire3;

        not NOT(wire2, A);
        xor EOR2(wire1, B, C);
        and AND2(wire3, wire1, A);
        nor NOR2(X, wire3, wire2);

endmodule

module testbench1_1;

        reg A, B, C;
        wire Z;
        circuit1_1 tb1(A,B,C,Z);

        initial
        begin
                A=1'b0; B=1'b0; C=1'b0;
                $display("Simulating output for circuit1_1");
                $monitor($time,,,"A = %b B = %b C = %b Z = %b",A,B,C,Z);

                #2 A=1'b0; B=1'b0; C=1'b1;
                #1 A=1'b0; B=1'b1; C=1'b0;
                #1 A=1'b0; B=1'b1; C=1'b1;
                #1 A=1'b1; B=1'b0; C=1'b0;
                #1 A=1'b1; B=1'b0; C=1'b1;
                #1 A=1'b1; B=1'b1; C=1'b0;
                #1 A=1'b1; B=1'b1; C=1'b1;
                #2 $finish;
        end

endmodule
```

Transcript

```
VSIM 7> run
# GetModuleFileName: The specified module could not be found.
#
#
# Simulating output for circuit1_1
#                    0  A = 0 B = 0 C = 0 Z = 0
#                    2  A = 0 B = 0 C = 1 Z = 0
#                    3  A = 0 B = 1 C = 0 Z = 0
#                    4  A = 0 B = 1 C = 1 Z = 0
#                    5  A = 1 B = 0 C = 0 Z = 1
#                    6  A = 1 B = 0 C = 1 Z = 0
#                    7  A = 1 B = 1 C = 0 Z = 0
#                    8  A = 1 B = 1 C = 1 Z = 1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Drill1_1.v(33)
#    Time: 10 ps  Iteration: 0  Instance: /testbench1_1
# 1
# Break in Module testbench1_1 at C:/intelFPGA_lite/20.1/Drill1_1.v line 33
```

Wave



Drill1_2

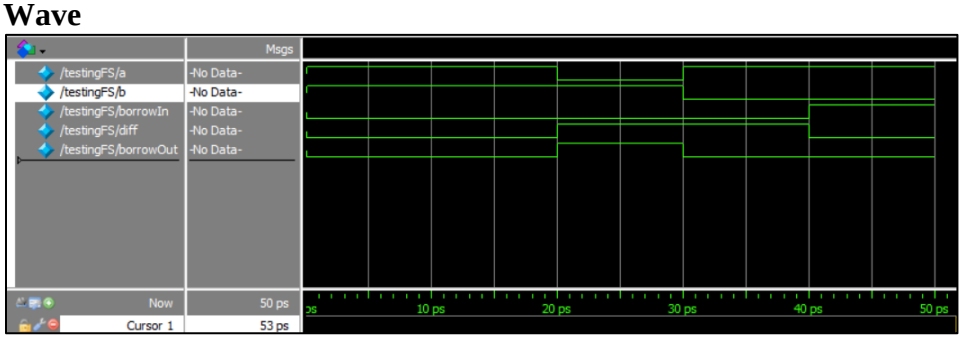
HDL Code

```
module full_subtract(diff,borrowOut,a,b,borrowIn);

        output  diff, borrowOut;
        input   a,b,borrowIn;
        assign  {borrowOut, diff} = a - b - borrowIn;

endmodule

module testingFS();

        reg     a,b,borrowIn;
        wire    diff,borrowOut;
        full_subtract fs(diff,borrowOut,a,b,borrowIn);

        initial begin

                a=1'b1; b=1'b1; borrowIn=1'b0;

                $display("a\tb\tbIn\tdiff\tbOut\ttime(ps)");
                $monitor("%b\t%b\t%b\t%b\t%b\t%0t",a,b,borrowIn,diff,borrowOut,$time);

                #10     a=1'b1;
                #10     a=1'b0; b=1'b1;
                #10     a=1'b1; b=1'b0;
                #10     borrowIn=1'b1;
                #10     $finish;

        end

endmodule
```

Transcript

```
VSIM 27> run
# GetModuleFileName: The specified module could not be found.
#
#
# a     b       bIn     diff    bOut    time(ps)
# 1     1       0       0       0       0
# 0     1       0       1       1       20
# 1     0       0       1       0       30
# 1     0       1       0       0       40
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Drilll_2.v(26)
#    Time: 50 ps  Iteration: 0  Instance: /testingFS
# 1
# Break in Module testingFS at C:/intelFPGA_lite/20.1/Drilll_2.v line 26
```

Wave



Descriptive Explanation

Drill1_1

The Verilog code defines a digital circuit in the circuit1_1 module, which uses NOT, XOR, AND, and NOR gates to compute the output X from inputs A, B, and C. The testbench1_1 module is used to simulate the circuit by providing various input combinations and monitoring the output. The testbench initializes the inputs, applies different values with specific delays, and observes the circuit's behavior. It uses $display to print a simulation header and $monitor to continuously track changes in input and output values. The simulation runs until it reaches a specified end time, ensuring the circuit operates correctly across all test cases.

Drill1_2

The code consists of a full subtractor module (full_subtract) and a testbench module (testingFS). The full_subtract module calculates the difference and borrow-out signals from two input bits (a and b) and an input borrow (borrowIn). The testingFS module simulates the full subtractor by applying different combinations of input values and observing the output signals. It uses $display to print the header and $monitor to continuously track and display the values of a, b, borrowIn, diff, borrowOut, and simulation time. The simulation runs through a series of input changes, ending after a set period with $finish.

Exercise1_1

The Verilog code defines a module exercise1_1 that uses NOT and NAND gates to produce a 4-bit output W from three inputs X, Y, and Z. Each bit of W is calculated using different combinations of the negated and non-negated inputs. The testbenchExercise1_1 module simulates the exercise1_1 circuit by applying various input values and monitoring the output. It initializes the inputs, applies changes at specific time intervals, and uses $monitor to display the simulation results in real-time. The simulation runs through all possible input combinations and concludes with $finish after a set period.

Exercise1_2

The code defines a module exercise1_2 that processes three inputs (A, B, C) and produces an output X using a combination of NOT, XOR, AND, and NOR gates. The testbenchExercise1_2 module simulates this circuit by applying different combinations of input values and displaying the results. It initializes the inputs, applies changes with specified delays, and uses $display to print the values of A, B, C, and X at each step. The simulation continues for a total time of 80000-time units, ending with $finish. The testbench ensures that the circuit behaves correctly across all input combinations.

Exercise1_3

The code defines a module exercise1_3 that computes two outputs, var1 and x_4, from three inputs (x_1, x_2, x_3) using XOR gates. The x_4 output is the XOR of all three inputs, while var1 is the XOR of the three inputs and x_4. The testbenchExercise1_3 module simulates this circuit by applying various input combinations and displaying the results. It initializes the inputs, changes them in a sequence with specific delays, and uses $monitor to continuously track and display the values of x_1, x_2, x_3, x_4, and var1. The simulation runs for a total of 8-time units before ending with $finish.